Sistemas de Comunicación y Seguridad Informática

MOMENTO 1: FUNDAMENTOS Y SEGURIDAD DE REDES

Semana 2 — Seguridad en Redes y Tráfico

Inspección de Paquetes con Wireshark, Puertos Inseguros

Mg. Gabriel Guerrero Arellano
Universidad Cooperativa de Colombia



SEMANA 2 — SEGURIDAD EN REDES Y TRÁFICO

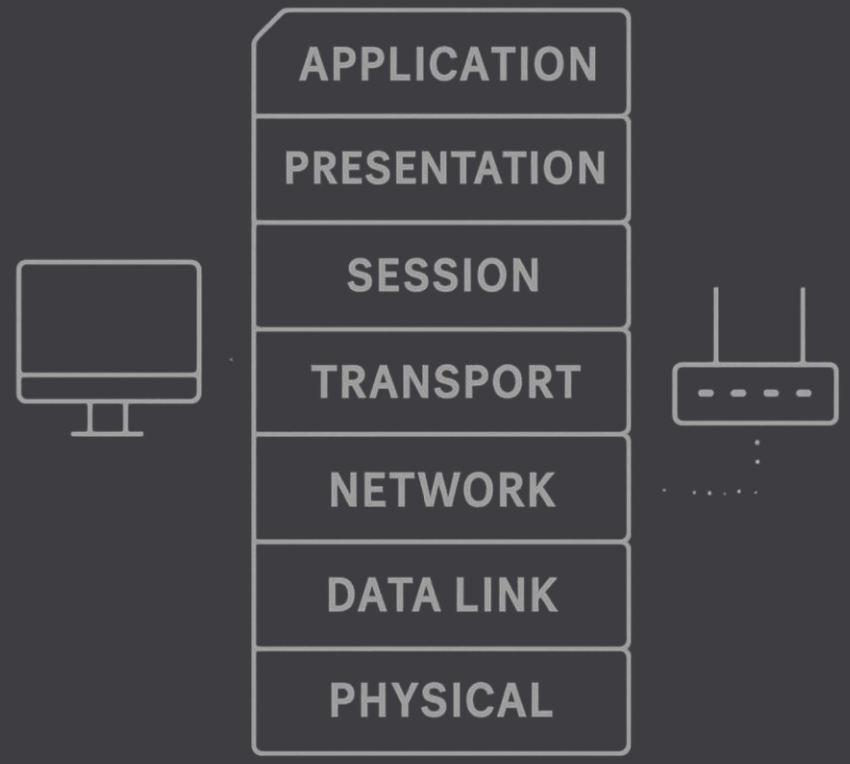
Sistemas de Comunicación y Seguridad Informática

Momento 1: Fundamentos y Seguridad de Redes

Tema Principal: Inspección de Paquetes con Wireshark, Firewalls y VPNs

Objetivo: Analizar el tráfico de red en vivo, identificar vulnerabilidades en protocolos sin cifrar y aplicar controles de seguridad defensivos.





Modelo OSI completo

Introducción al Estándar ISO

¿Qué es el Modelo OSI?

Es un modelo de referencia creado en 1980 que estandariza la comunicación entre sistemas diversos para garantizar la interoperabilidad global.

El fin de los sistemas propietarios

Su implementación permitió que redes de distintos fabricantes pudieran intercambiar información sin impedimentos técnicos o de software.



7 APLICACIÓN

Proporciona servicios de red directos al usuario (como cerreo SMTP o transferencia FTP) y APIs de alto nivel.

Unidad de datos (PDU): Datos



6 PRESENTACIÓN

Traduce, eifra y comprime la información para que sea legible entre sistemas con dicöntas sintaxis.

Unidad de datos (PDU): Datos



5 SESIÓN

Estableca, administra y finaliza las sesiones de comunicación entre aplicaciones losales y remotas.

Unidad de datos (PDU): Datos



4 TRANSPORTE

Garentiza una transmisión confiable y libre de erroros entre extremos mediante protocolos como TCP y UDP.

Unidad de datos (PDU): Segmento / Detagrama



3 RED

Se encarga del direccionamiento lógico (direcciones IP) y el enrutamiento de datos a través de múltiples redes.

Unidad de datos (PDU): Paquete



2 ENLACE DE DATOS

Gestiona el direccionamiento físico (direcciones MAC) y el tráncito contiable sobre el enlace físico mediante Switches.

Unidad de datos (PDU): Trama



1 FÍSICA

Define las especificaciones eléctricas y mecánicas para la transmisión de tifla a través de cablos, ffora óptica o señalos de radio.

Unidad de datos (PDU): Bit / Beudios

El Modelo OSI y la Inspección de Tráfico

¿En qué capas operamos durante el análisis de red?

1

Capa 7 — Aplicación

Protocolos HTTP, DNS, SSH, FTP — donde residen los datos del usuario y las credenciales de acceso.

2

Capa 4 — Transporte

Control de flujo TCP/UDP y gestión de puertos (ej. 80, 443, 22). Aquí se establece el handshake de conexión.

3

Capa 3 — Red

Direccionamiento IP origen y destino para el enrutamiento de paquetes entre redes distintas.

4

Capa 2 — Enlace de Datos

Identificación física mediante direcciones MAC en el segmento local de red.



Conceptos Fundamentales de Captura

Mecanismos clave para la intercepción de paquetes

Modo Promiscuo

Promiscuous Mode

Configuración de la tarjeta de red para capturar todo el tráfico del medio físico, no solo el propio. Permite la visibilidad completa del segmento de red.

Inspección Profunda de Paquetes

Deep Packet Inspection — DPI

Análisis detallado del contenido o payload para detectar fuga de datos sensibles o presencia de malware en el tráfico.

Ataques Man-In-The-Middle

MITM

Intercepción de comunicaciones entre dos nodos no conscientes de la intrusión. El atacante puede leer y modificar el tráfico en tránsito.



Introducción a Wireshark

La herramienta estándar en la industria para el análisis de protocolos

Funcionalidad

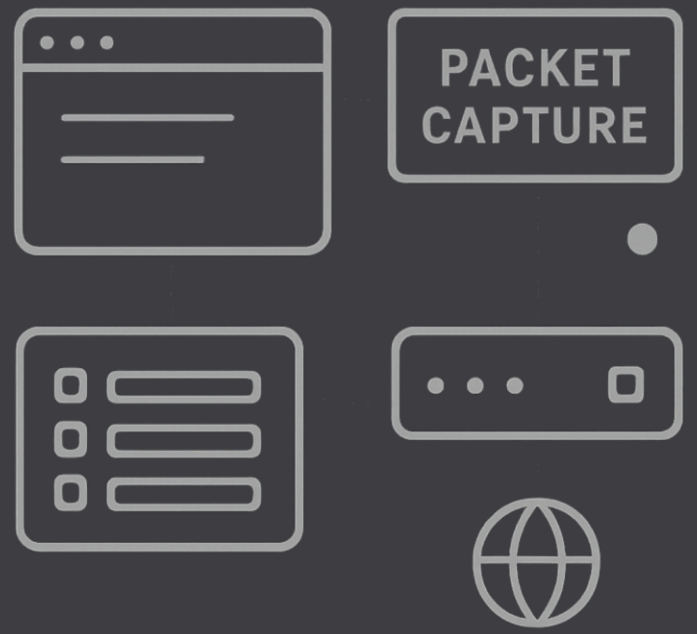
Captura y analiza paquetes que viajan por interfaces Wi-Fi o Ethernet en tiempo real. Permite reconstruir sesiones completas y examinar cada byte transmitido.

Uso Defensivo

Auditoría de red, diagnóstico de fallas de conectividad y detección de anomalías de seguridad. Herramienta esencial para equipos de respuesta a incidentes.

Soporte de Protocolos

Compatible con cientos de protocolos de red modernos: HTTP, HTTPS, DNS, FTP, SSH, SMTP, y muchos más. Disección automática por capas OSI.



Estructura de la Interfaz de Wireshark

Tres paneles principales para el diagnóstico visual

1

Panel Hexadecimal — Bottom

Volcado directo de los bytes en estado bruto (raw data). Permite inspección a nivel de bit de cada paquete capturado.

2

Panel de Detalles del Paquete — Middle

Desglose jerárquico por capas del modelo OSI. Expande cada encabezado de protocolo para análisis granular.

3

Panel de Lista de Paquetes — Top

Muestra el historial cronológico de paquetes: Número, Tiempo, Origen, Destino, Protocolo, Info. Vista general del tráfico capturado.



Sintaxis de Filtros de Visualización en Wireshark

Aislando el tráfico relevante del ruido de la red

Filtro por Protocolo

```
http
dns
```

Muestra únicamente el tráfico del protocolo especificado, eliminando todo el ruido restante de la captura.

Filtro por Dirección IP

```
ip.addr == 192.168.1.50
```

Filtra todo el tráfico entrante y saliente asociado a una dirección IP específica en la red local.

Filtro por Puerto

```
tcp.port == 80 || tcp.port == 443
```

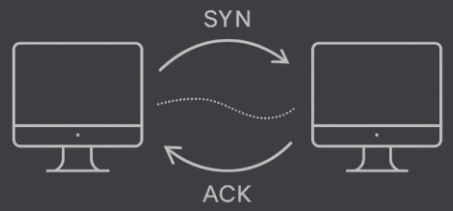
Captura el tráfico web estándar (HTTP y HTTPS) usando el operador lógico OR para múltiples puertos.

Filtro por Métodos HTTP

```
http.request.method == "POST"
```

Aísla únicamente las peticiones POST, donde típicamente se transmiten credenciales y datos de formularios.





El Handshake de 3 Vías TCP

Establecimiento de conexión segura en Capa 4 antes de transferir datos

UNIVERSIDAD COOPERATIVA DE COLOMBIA

1

Paso 1 — SYN

El cliente envía una petición de sincronización con un número de secuencia inicial aleatorio al servidor destino.

2

Paso 2 — SYN-ACK

El servidor responde confirmando la recepción (ACK) y solicita sincronización con su propio número de secuencia.

3

Paso 3 — ACK

El cliente confirma la conexión. La sesión queda oficialmente establecida y comienza la transferencia de datos.

Filtro en Wireshark: `tcp.flags.syn == 1` — Captura todos los paquetes SYN para visualizar el inicio de cada conexión TCP en la red.

Análisis Comparativo: HTTP vs HTTPS

Riesgos de seguridad en protocolos sin cifrado frente a canales protegidos

HTTP — Texto Plano ⚠

- Transmisión Expuesta: El payload incluye las credenciales legibles en texto claro, sin ninguna protección.
- Estructura Visible: Encabezados y cuerpo de la petición visibles para cualquiera en la red local.
- Riesgo Crítico: Captura directa de formularios mediante inspección simple de datos HTTP POST.

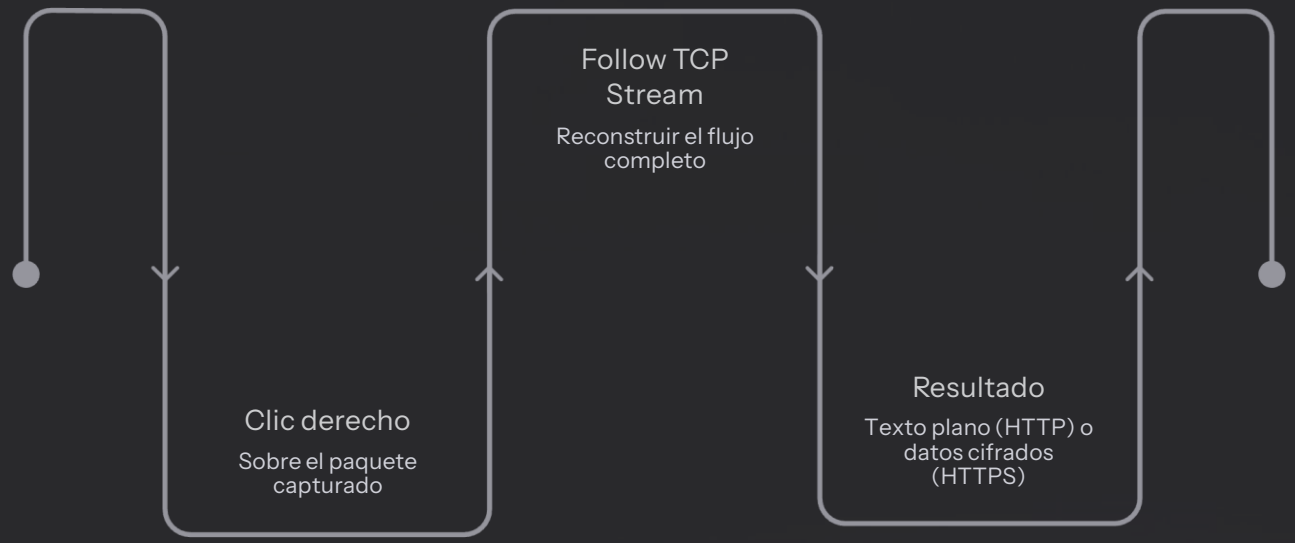
HTTPS — Cifrado TLS/SSL ☑

- Handshake TLS: Negociación previa de claves criptográficas entre cliente y servidor antes de transmitir datos.
- Payload Cifrado: Transforma el tráfico legible en un flujo de datos ininteligible para interceptores.
- Seguridad: Inmunidad contra ataques pasivos de escucha en redes Wi-Fi públicas no confiables.



Demostración en Vivo: TCP Stream en Wireshark

Reconstrucción del flujo completo de una sesión



Acción

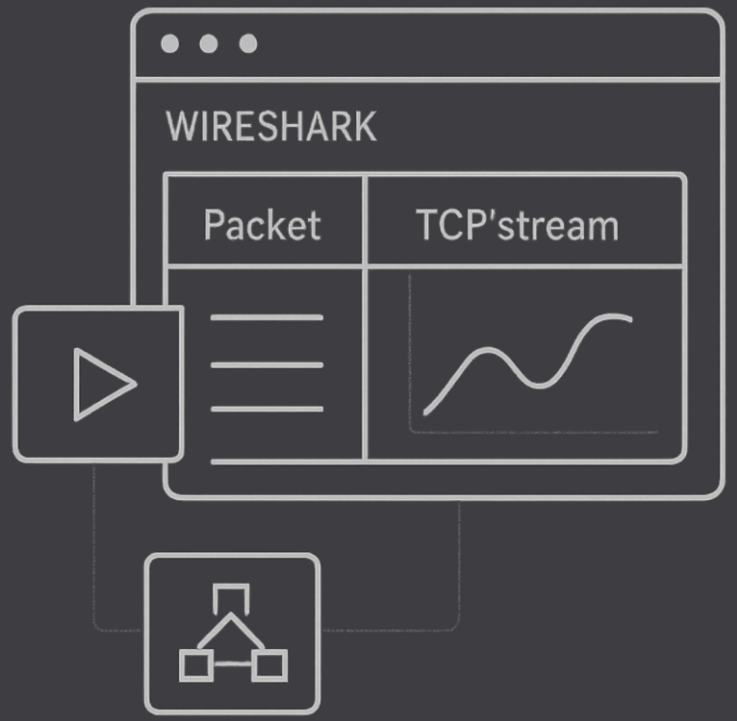
Clic derecho sobre el paquete capturado → Follow → TCP Stream. Wireshark reconstruye la sesión completa en una ventana dedicada.

Resultado HTTP

Muestra la conversación completa de la petición y respuesta en texto plano. Credenciales, cookies y datos de formulario completamente expuestos.

Resultado HTTPS

Muestra únicamente caracteres encriptados e ilegibles. El cifrado TLS protege efectivamente el contenido de la sesión.



Firewalls: La Primera Línea de Defensa

Mecanismos de control de acceso a nivel de red

Firewalls Sin Estado

Stateless

Evalúan paquetes de forma aislada basándose en reglas fijas de IP y Puerto. Rápidos pero sin contexto de sesión.

Firewalls Con Estado

Stateful

Realizan seguimiento a la sesión completa para autorizar solo el tráfico de respuesta legítimo. Mayor seguridad contextual.

Acciones de Regla

Tres decisiones posibles por paquete:

- ACCEPT: Permitir el tráfico
- DROP: Descartar en silencio
- REJECT: Rechazar informando al emisor



Configuración de Firewalls en Linux

Comandos prácticos para la gestión de seguridad de tráfico — `iptables` / `UFW`

Ver Reglas Activas

```
sudo iptables -L -v -n
```

Lista todas las cadenas y reglas activas con contadores de paquetes y bytes. La opción `-n` evita la resolución DNS para mayor velocidad.

Bloquear Puerto HTTP (80)

```
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

Agrega una regla al final de la cadena INPUT para descartar silenciosamente todo el tráfico TCP entrante al puerto 80.

Permitir SSH Seguro (22)

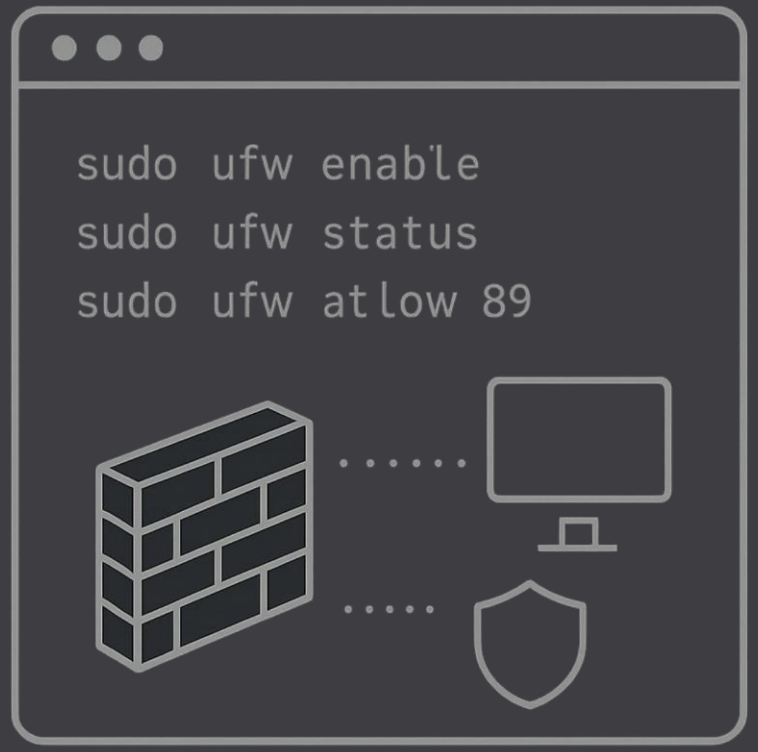
```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Autoriza explícitamente las conexiones SSH entrantes para mantener el acceso administrativo remoto al servidor.

Gestión Simplificada con UFW

```
sudo ufw deny 80/tcp && sudo ufw enable
```

UFW (Uncomplicated Firewall) ofrece una interfaz simplificada sobre iptables. Ideal para administración rápida en entornos Ubuntu/Debian.



Redes Privadas Virtuales (VPNs)

Aislamiento y cifrado sobre infraestructura pública

Túnel Cifrado

Encapsula todo el tráfico IP del dispositivo dentro de un canal protegido hacia un servidor remoto. El tráfico es invisible para observadores en la red local.

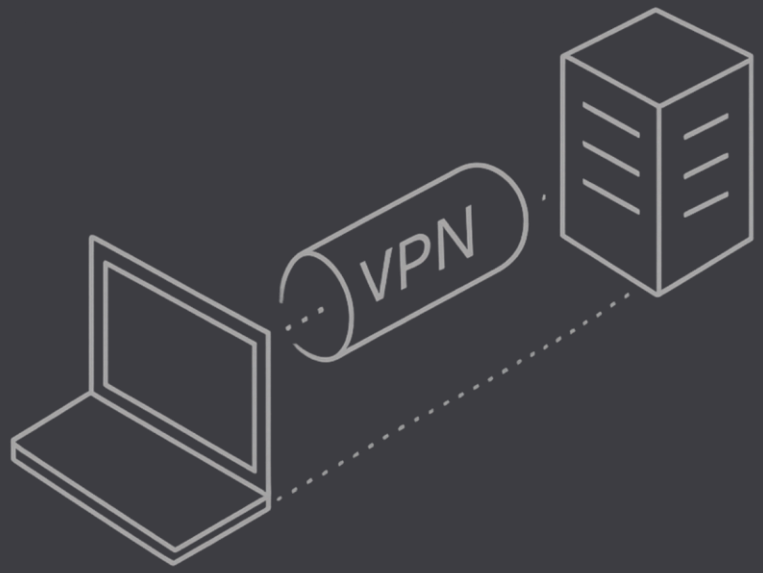
Protocolos Modernos

Tres estándares principales en uso activo:

- OpenVPN: Código abierto, altamente configurable
- IPsec: Estándar empresarial robusto
- WireGuard: Moderno, rápido y minimalista

Beneficios de Seguridad

Confidencialidad en redes no confiables, ocultamiento de IP origen y evitación de la inspección local por parte de ISPs o administradores de red.



Lab Parte 1 — Herramientas e Instalación (Windows)

Preparación del entorno de trabajo local en Windows

HERRAMIENTAS REQUERIDAS

- Wireshark para Windows
- Git Bash
- PowerShell (como Administrador)
- Navegador Web (Chrome/Edge/Firefox)

Wireshark

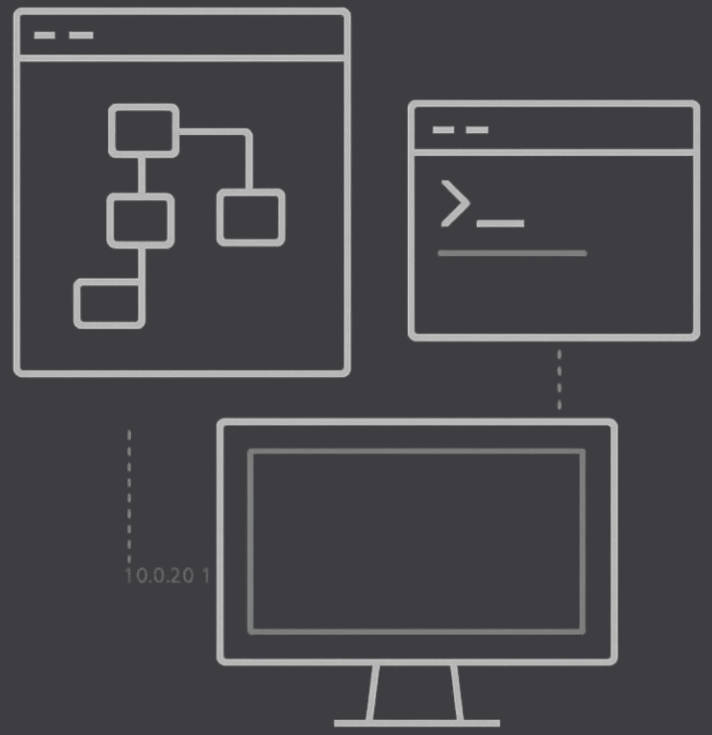
Abrir la aplicación ejecutable como Administrador (Clic derecho → Ejecutar como Administrador) para habilitar el driver de captura.

Git Bash

Abrir la consola para levantar el servidor web local en 1 segundo.

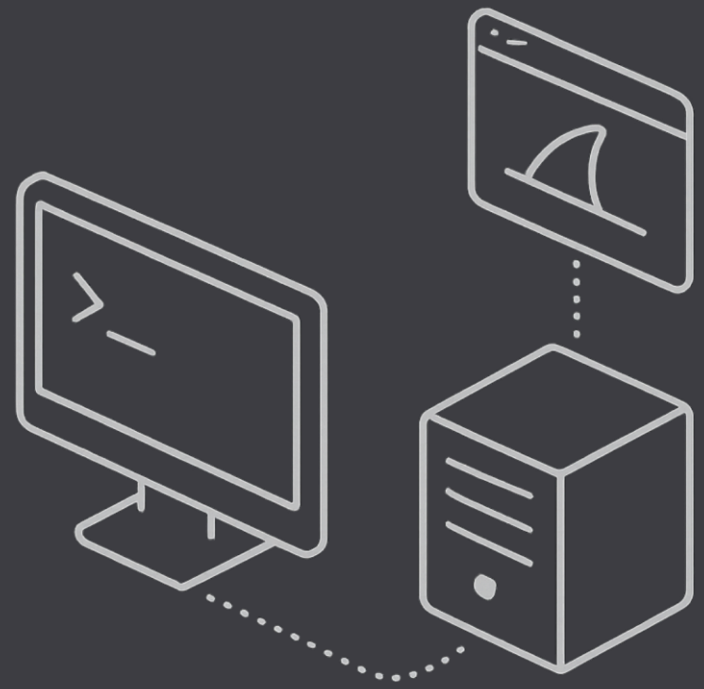
Sin dependencia de Internet

Al usar un servidor local, la práctica funciona sin consumir datos móviles ni depender de redes externas.



Lab Parte 2 — Despliegue del Servidor HTTP y Captura

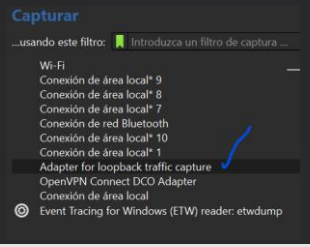
Generación de tráfico inseguro (HTTP) en la máquina local



01

Paso 1: Iniciar captura en Wireshark

En la pantalla de inicio de Wireshark, seleccionar la interfaz "Adapter for loopback traffic capture“ para el ejercicio con servidor local (o Wi-Fi) con doble clic.



02

Paso 2: Levantamiento del servidor HTTP local

Abrir Git Bash y ejecutar el siguiente comando:

```
python -m http.server 8080
```

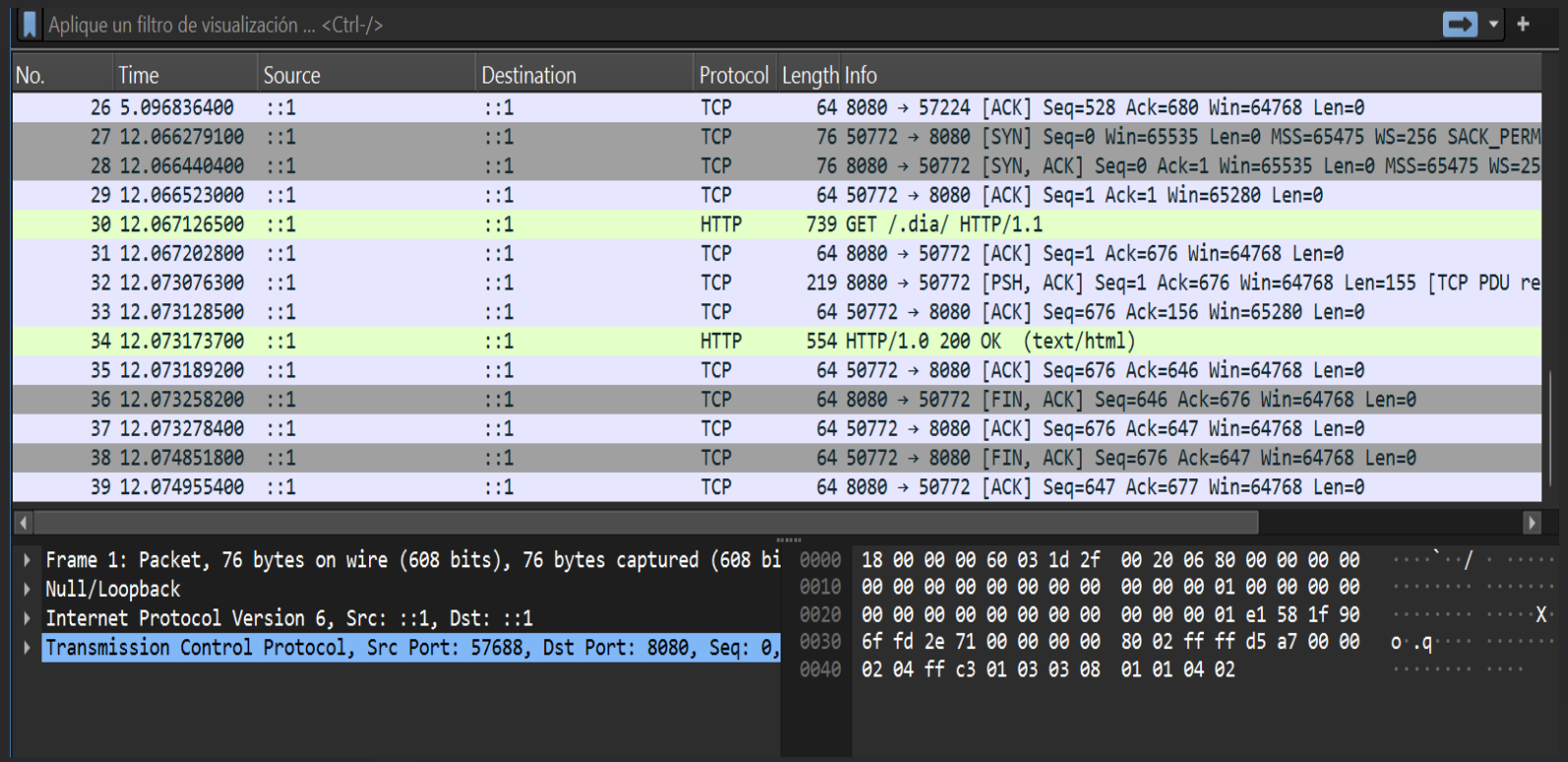
03

Paso 3: Generar peticiones get HTTP

Abrir el navegador e ingresar a la dirección: http://localhost:8080

Navegar o hacer clic entre los archivos/carpetas mostrados para generar peticiones HTTP en texto plano. Esto sirve para producir tráfico de red en vivo, permitiendo observen en Wireshark cómo cada interacción en la web transmite datos (encabezados, rutas y parámetros) en texto plano a través de la Capa de Aplicación.

Lab Parte 2 — Significado Pantalla, que hacemos?



Al hacer clic en la carpeta .dia/, el navegador envió la petición **GET /.dia/ HTTP/1.1** (Paquete 30, resaltado en verde). El servidor de Python la procesó y respondió con **HTTP/1.0 200 OK** (Paquete 34, resaltado en verde), entregando la lista de archivos en código HTML. Todo este proceso fue coordinado previamente por el establecimiento de la conexión TCP (paquetes 27 al 29).

Que muestra la interface de wireshark:

- **Panel Superior (Lista de Paquetes / *Packet List*):**
- **No.:** Número correlativo del paquete capturado.
- **Time:** Tiempo en segundos transcurrido desde que inició la captura.
- **Source / Destination:** ::1 representa la dirección IP local en formato **IPv6** (Loopback / localhost).
- **Protocol:** El protocolo de la capa correspondiente (**TCP** para la infraestructura de transporte y **HTTP** para la capa de aplicación).
- **Length:** Tamaño del paquete en bytes.
- **Info:** Detalle de la operación (Handshake TCP [SYN], [SYN, ACK], [ACK], Petición HTTP GET, o Respuesta HTTP 200 OK).
- **Panel Inferior Izquierdo (Detalles del Paquete / *Packet Details*):**
- Desglosa las capas del modelo OSI/TCP-IP del paquete seleccionado (Capa física Frame, Capa de enlace Loopback, Capa de red IPv6, Capa de transporte TCP).

Lab Parte 3 — Filtrado e Inspección Profunda (DPI)

Reconstrucción de tráfico HTTP en texto claro con Wireshark

01

Paso 1: Aplicar filtro de Puerto local en Wireshark

En la barra superior de filtros de Wireshark, escribir el siguiente filtro y presionar Enter:

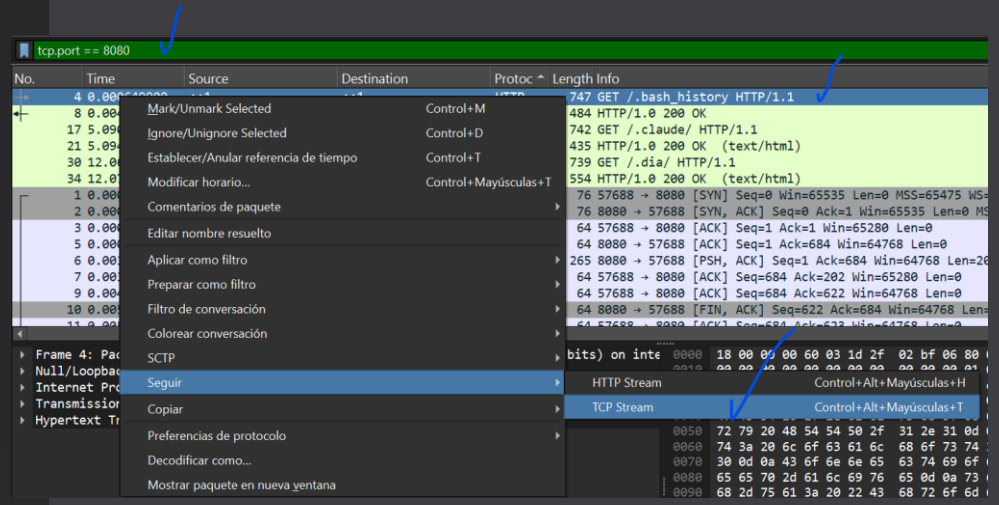
```
tcp.port == 8080
```

02

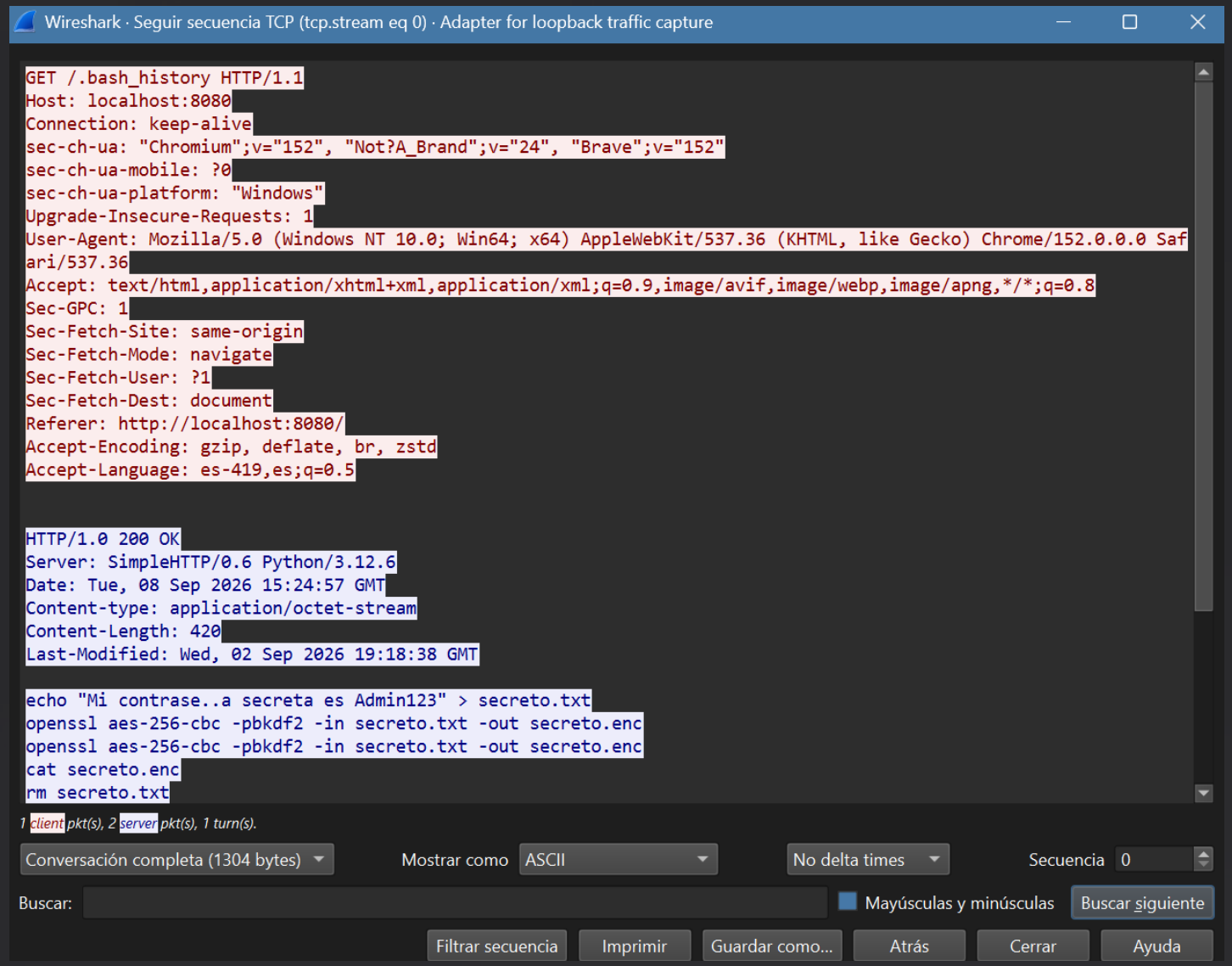
Paso 2: Analizar el payload del paquete

Ubicar un paquete con protocolo HTTP (petición GET /). Dar clic derecho sobre el paquete → Follow → TCP Stream.

> Resultado Obtenido: Se abrirá la ventana emergente mostrando los encabezados HTTP puros (Host: localhost:8080, User-Agent, etc.) en texto totalmente legible y sin cifrar.



Lab Parte 3 — Vulnerabilidades



Wireshark reconstruyó la conversación TCP completa codificando los roles por colores:

- **Texto en Rojo (Petición del Cliente / Navegador):** El navegador (Brave/Chromium en Windows) hizo una petición GET /.bash_history al servidor.
- **Texto en Azul (Respuesta del Servidor Python):** El servidor local respondió con un código HTTP/1.0 200 OK (éxito) y le envió directamente el contenido del archivo solicitado (.bash_history).

¿Qué significa?

1.Fuga de Información Crítica (Falta de Cifrado):
Cualquier persona o atacante que esté inspeccionando la red puede leer en texto plano tanto los encabezados de la petición como los datos del servidor.

2.Impacto de Seguridad:
Al solicitar el archivo .bash_history por HTTP no seguro, el servidor entregó el historial de comandos ejecutados en la máquina, dejando expuesta en texto puro la contraseña y los comandos del usuario:
Plaintext
echo "Mi contrase..a secreta es Admin123" > secreto.txt

3.Conclusión Pedagógica para la Clase:
Esto demuestra por qué **HTTP plano es peligroso**: no solo no cifra las contraseñas enviadas en formularios, sino que permite que un atacante capture cualquier archivo o dato sensible que viaje por la red **sin protección TLS/HTTPS.**

Lab Parte 4 — Interrupción del Servicio y Verificación

Interrupción del servicio expuesto y comprobación de indisponibilidad

01

Paso 1: Interrupción del servicio inseguro

Ir a la terminal de Git Bash donde está ejecutándose el servidor HTTP local. Presionar el atajo de teclado:

```
Ctrl + C
```

02

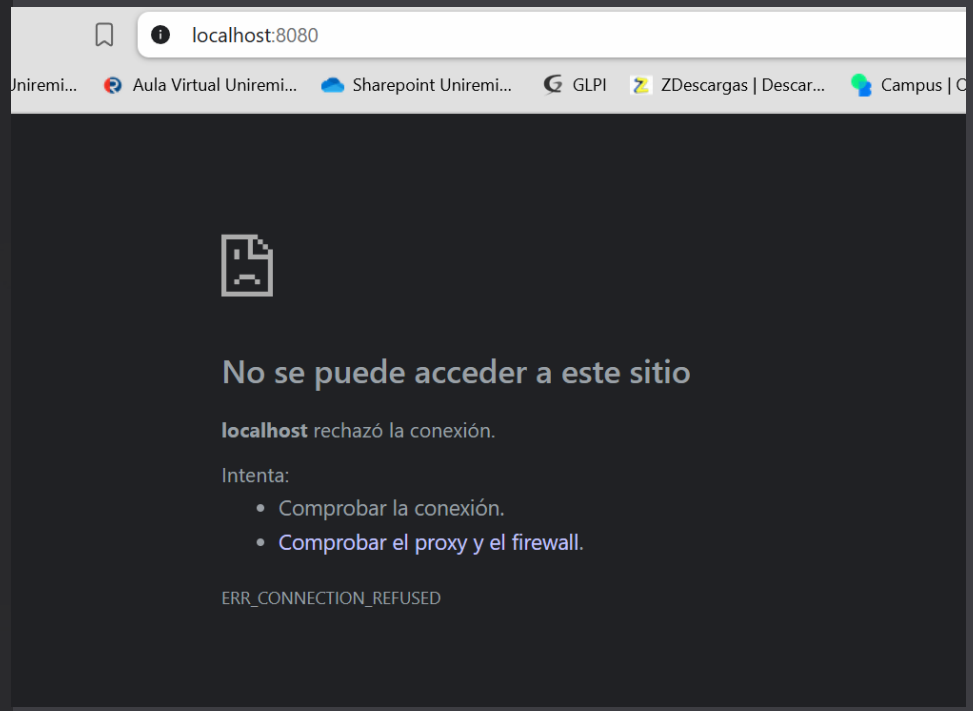
Paso 2: Verificación de la mitigación en el navegador

Volver al navegador y recargar la página (http://localhost:8080).

Resultado esperado: El navegador rechazará la conexión desplegando el mensaje ERR_CONNECTION_REFUSED.

Evidencia a Entregar: Una captura de pantalla donde se observe la ventana de TCP Stream en Wireshark (fuga de datos) junto al navegador mostrando la conexión rechazada.

> Cierre del Taller: “La mitigación básica cuando detectamos un puerto inseguro sin cifrado exponiendo información es dar de baja el servicio de inmediato (Ctrl + C) para evitar la fuga de datos mientras se implementa un canal seguro con TLS/HTTPS."
>
> — Mg. Gabriel Guerrero Arellano



Conclusiones y Diagnóstico de Errores

Puntos clave y buenas prácticas de análisis

Visibilidad Total

La inspección de paquetes es fundamental para validar si las políticas de seguridad se cumplen en la práctica. Lo que no se mide, no se puede proteger.

Uso Obligatorio de TLS

Ninguna aplicación en producción debe transmitir credenciales sobre HTTP plano. El cifrado TLS es un requisito mínimo de seguridad, no una opción.

Solución de Problemas en Lab

Wireshark no detecta interfaces

→ Ejecutar con permisos de administrador (`sudo wireshark` en Linux o "Ejecutar como administrador" en Windows). Sin privilegios elevados, el modo promiscuo no está disponible.

Exceso de tráfico en la captura

→ Filtrar por la IP de destino específica usando `ip.addr == [IP_OBJETIVO]` para reducir el ruido y enfocarse en el tráfico relevante del ejercicio.



SEMANA 2 — CIERRE DE SESIÓN

Resumen de la Sesión

Sistemas de Comunicación y Seguridad Informática — Momento 1

01

Modelo OSI y Captura

Identificamos las capas relevantes (2, 3, 4, 7) para el análisis de tráfico y los mecanismos de intercepción: modo promiscuo, DPI y MITM.

02

Wireshark y Filtros

Dominamos la interfaz de tres paneles y la sintaxis de filtros de visualización para aislar tráfico HTTP, DNS, IP y TCP específico.

03

HTTP vs HTTPS

Demostramos empíricamente la exposición de credenciales en texto plano sobre HTTP y la protección efectiva que ofrece TLS/SSL.

04



Mg. Gabriel Guerrero Arellano
Universidad Cooperativa de Colombia — Sistemas de Comunicación y Seguridad Informática